

Inteligencia Artificial Generativa Aplicada al Análisis de
Datos

Tema 2. Modelos generativos aplicados a datos. Uso de GAN, VAE y transformers en la generación de datos

Índice

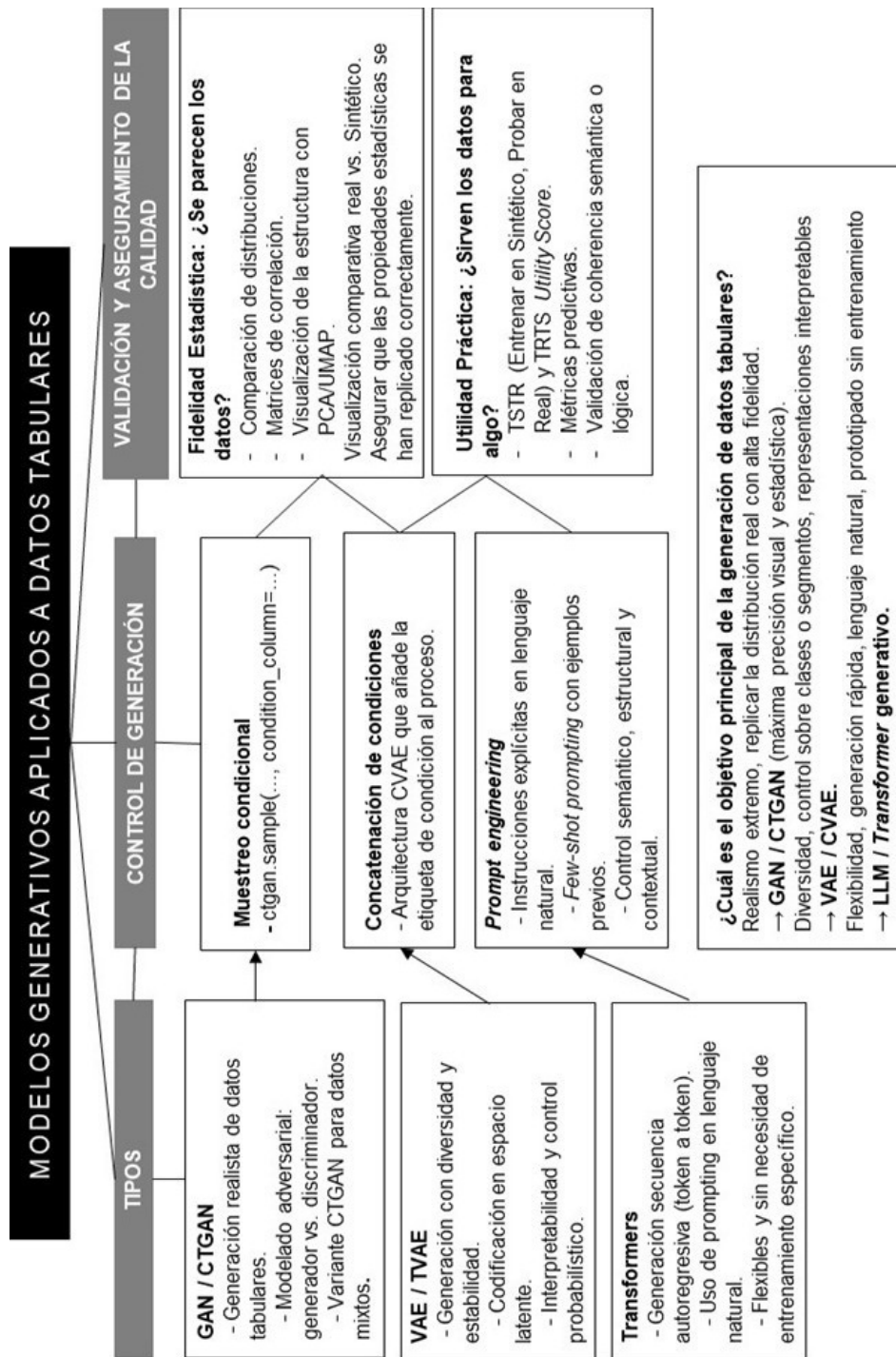
Esquema

Ideas clave

- 2.1. Introducción y objetivos
- 2.2. Arquitectura y entrenamiento de GAN
- 2.3. VAE para datos tabulares
- 2.4. Transformers generativos y LLM para datos estructurados
- 2.5. Comparación de modelos: usos, ventajas y desventajas
- 2.6. Modelos condicionales y control de generación
- 2.7. Evaluación de calidad de datos generados
- 2.8. Exploración práctica
- 2.9. Referencias bibliográficas

A fondo

- SDV: Welcome to the SDV!
- Synthetic Data Generation with CTGAN – Step-by-Step Tutorial
- Synthetic data generation using large language models



2.1. Introducción y objetivos

En el tema, Fundamentos de IA generativa, se establecieron algunos de los fundamentos conceptuales de la inteligencia artificial generativa, analizando algunas de sus principales arquitecturas, como GAN, VAE y transformadores, y describiendo sus capacidades en múltiples dominios. En este segundo tema nos centraremos en una de sus aplicaciones más relevantes y operativas dentro del análisis de datos: la generación sintética de datos estructurados.

Los datos estructurados —caracterizados por seguir un esquema bien definido de variables y estar dispuestos habitualmente en filas y columnas, como en hojas de cálculo o bases de datos relacionales— constituyen la base de la mayoría de los procesos analíticos en entornos empresariales y regulatorios. Sin embargo, su obtención enfrenta obstáculos como restricciones legales, costes de adquisición, escasez en ciertos dominios o problemas de privacidad. Ante este escenario, la generación sintética de datos estructurados emerge como una solución estratégica. Las técnicas de IA generativa permiten simular distribuciones multivariantes complejas y sintetizar datos realistas, preservando patrones y relaciones internas. Esto posibilita ampliar conjuntos de entrenamiento sin comprometer datos sensibles, diseñar entornos de prueba replicables y controlados, simular escenarios de negocio en condiciones hipotéticas y mitigar sesgos de muestreo o desbalanceo en variables críticas, entre otras aplicaciones.

A lo largo de este tema, exploraremos cómo se adaptan y entrenan las principales arquitecturas generativas para este propósito, destacando algunas de sus ventajas, limitaciones y condiciones óptimas de uso. Además, se analizarán las métricas y metodologías para evaluar la calidad de los datos generados, con una orientación práctica.

Al finalizar este tema, el estudiante será capaz de:

- ▶ Objetivo número 1: comprender conceptual y operacionalmente los modelos generativos para la creación de datos estructurados.
- ▶ Objetivo número 2: evaluar la calidad de los datos generados mediante métricas adecuadas.
- ▶ Objetivo número 3: comparar arquitecturas generativas para seleccionar la más idónea en contextos de análisis de datos reales.

2.2. Arquitectura y entrenamiento de GAN

Las Redes Generativas Antagónicas (GAN) introducidas por Goodfellow *et al.* (2014), constituyen una de las arquitecturas más influyentes en el campo del aprendizaje generativo. Su capacidad para aprender distribuciones complejas directamente a partir de datos observados las ha convertido en una herramienta clave para la generación de contenido sintético, desde imágenes hasta, más recientemente, datos tabulares estructurados.

Profundizando un poco más en lo estudiado en el tema titulado «Fundamentos de IA Generativa», la arquitectura básica de una GAN se sustenta en un proceso competitivo entre dos redes neuronales profundas entrenadas simultáneamente:

1. **Generador (G):** toma como entrada un vector aleatorio, por ejemplo $z \sim N(0, I)$, y lo transforma en una muestra sintética $G(z)$ que busca imitar los datos reales. A este lo llamaremos el «falsificador».
2. **Discriminador (D):** Actúa como un **detective**. Su trabajo es recibir muestras tanto de los datos reales como de las **falsificaciones** del Generador (sintéticas), y debe aprender a distinguir unas de otras, etiquetándolas como **Real o Falsa**.

Este proceso se formaliza como un juego min-max con la siguiente función objetivo:

$$\min_G \max_D V(D, G) = E_{x \sim P_{data}} [\log D(x)] + E_{z \sim z} [\log (1 - D(G(z)))]$$

Con E la esperanza respecto a datos reales o generados y con $D(\bullet)$ la probabilidad de que el discriminador asigne mayor verosimilitud a que x sea un dato real que a que un dato generado lo sea, el objetivo final es alcanzar un equilibrio en el que el discriminador no pueda diferenciar los datos reales de los sintéticos mejor que por azar, lo cual indica que el generador ha aprendido la distribución de los datos reales.

Aunque las GAN clásicas fueron diseñadas inicialmente para datos continuos y estructurados, como imágenes, su aplicación directa a datos tabulares presenta retos particulares: la mezcla de variables continuas y categóricas, la alta cardinalidad de variables discretas y dependencias no lineales complejas y multivariadas.

Para abordar estos desafíos, se han desarrollado variantes especializadas, como CTGAN (Conditional Tabular GAN), propuesta por Xu *et al.* (2019), que introduce técnicas específicas: la codificación condicional de variables categóricas (representar categorías como entradas explícitas para guiar al generador, por ejemplo, con one-hot o embeddings); el entrenamiento por mode-specific sampling para evitar el colapso de modo (muestrear por subgrupos o clases para asegurar que la GAN aprenda todos los modos —«tipos»— de los datos y evitar que se centre solo en unos pocos, ignorando la diversidad de los datos reales); y el rediseño de la arquitectura y de la función de pérdida para mejorar la generación discreta, esto es, adaptar la red y su objetivo para manejar mejor datos discretos (como categorías), que no son naturalmente generados por salidas continuas estándar (Guja *et al.*, 2025).

CTGAN gestiona la estructura tabular de tipo mixto con mayor robustez que las GAN estándar, lo que garantiza realismo y variedad en los registros generados (Guja *et al.*, 2025).

El siguiente resumen sintetiza los componentes clave para aplicar correctamente una arquitectura GAN en el dominio tabular:

Elemento	Descripción
Variables de entrada	<i>Dataset</i> con estructura tabular: variables categóricas y continuas.
Variable objetivo	No necesaria, ya que las GAN son no supervisadas. Puede incorporarse de forma condicional si se desea controlar la salida (CTGAN).
Transformaciones requeridas	1. Normalización de variables numéricas. 2. Codificación categórica (<i>one hot</i> o <i>embeddings</i>) 3. Manejo de valores nulos.
Modelo recomendado	CTGAN para variables mixtas; variantes personalizadas en PyTorch o TensorFlow si se requieren arquitecturas más específicas.
Herramientas / software	1. Desarrollado como parte del proyecto Synthetic Data Vault (SDV) por el MIT Data to AI Lab (MIT Data to AI Lab, 2023). Esta biblioteca permite generar datos sintéticos de forma flexible con configuraciones adaptadas a variables mixtas. 2. En casos donde se requiera modificar la arquitectura, personalizar la función de pérdida o adaptar el entrenamiento a características específicas del dominio, pueden utilizarse <i>frameworks</i> como PyTorch o TensorFlow, siguiendo las directrices de implementación propuestas por Goodfellow et al. (2016) o Foster (2019).
Tipo de salida esperada	Nuevos registros sintéticos que preservan la estructura, relaciones internas y características estadísticas de los datos reales.

Tabla 1. Elementos necesarios para aplicar GAN a datos estructurados. Fuente: elaboración propia.

2.3. VAE para datos tabulares

Los autoencoders variacionales (VAE) introducidos por Kingma y Welling (2014) son modelos generativos basados en redes neuronales que combinan ideas de aprendizaje profundo y modelado probabilístico bayesiano. A diferencia de los autoencoders clásicos, los VAEs no solo reconstruyen datos, sino que aprenden una distribución latente que puede utilizarse para generar nuevas muestras.

En el contexto de generación de datos estructurados, los VAEs (autoencoders variacionales), ofrecen una alternativa más estable y fácil de interpretar que las GAN. Su capacidad para aprender una representación latente continua y bien organizada permite no solo generar nuevos datos, sino también explorar, modificar o controlar características específicas de las muestras generadas, como ajustar atributos individuales (por ejemplo, edad, tamaño, categoría), o realizar interpolaciones suaves entre ejemplos para entender las transiciones entre clases o perfiles. Esta estructura latente facilita tareas como la interpolación entre ejemplos o la manipulación dirigida de atributos, lo que los hace especialmente útiles cuando se requiere comprensión del proceso generativo o generación controlada.

Gracias a esta representación latente es posible, por ejemplo, navegar por el espacio de variables ocultas para observar cómo pequeños cambios afectan al resultado generado; o bien fijar determinadas características mientras se varían otras. Esto resulta valioso en dominios donde se desea mantener el control semántico del dato simulado, como en la generación de perfiles sintéticos en seguros, registros clínicos o configuraciones de productos, lo que permite generar datos nuevos que respeten propiedades conocidas o deseadas.

El VAE consta de dos componentes entrenados conjuntamente.

1. **Codificador** $q_\phi(z|x)$: toma una muestra de entrada x (por ejemplo, una fila de datos) y produce una distribución sobre una variable latente z , normalmente una gaussiana multivariada.
2. **Decodificador** $p_\theta(x|z)$: toma una muestra z del espacio latente y trata de reconstruir x a partir de ella.

El entrenamiento maximiza la función conocida como evidencia inferior variacional (ELBO).

$$\mathcal{L}(\theta, \phi, x) = E_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z))$$

Este enfoque promueve una codificación estructurada y continua en el espacio latente, lo que permite muestrear nuevas instancias sintéticas. El primer término (esperanza) indica que el VAE reconstruya correctamente los datos originales, y el segundo término (KL) obliga al codificador a producir vectores z que sigan una forma conocida (p. ej. normal estándar), para que podamos muestrear nuevos z y generar datos. En resumen: se busca aprender una representación latente útil (continua, suave) que permita generar datos similares a los reales, mientras se mantiene el control sobre el espacio latente.

Los VAE son particularmente útiles en ámbitos donde se requiere una manipulación interpretable del espacio latente y la estabilidad del entrenamiento es una preocupación (Foster, 2019; Ramchandani, 2025). Sin embargo, aunque fueron diseñados originalmente para imágenes, pueden adaptarse a datos estructurados si se abordan adecuadamente las particularidades de los datos tabulares, como variables mixtas (categóricas y numéricas), el escalamiento y la codificación y las dependencias multivariantes y las relaciones complejas.

Existen implementaciones específicas, como VAE (tabular variational autoencoder) y SDV (MIT Data to AI Lab, 2023) que ajustan la arquitectura y la función de pérdida para este tipo de datos.

La siguiente tabla 2 sintetiza los componentes clave para aplicar correctamente una arquitectura VAE en datos tabulares.

Elemento	Descripción
Variables de entrada	Datos tabulares con variables numéricas y categóricas.
Variable objetivo	No requerida. Modelo no supervisado.
Transformaciones requeridas	- Normalización. - <i>One hot encoding</i> o <i>embeddings</i>. - Imputación de valores nulos.
Modelo recomendado	TVAE (Tabular VAE); también se pueden construir VAEs personalizados en PyTorch o TensorFlow.
Herramientas / software	- <code>sdv.tabular.TVAE</code> (SDV) - Implementaciones customizadas con PyTorch o TensorFlow.
Tipo de salida esperada	Datos sintéticos estadísticamente similares a los reales.
Principales ventajas	Entrenamiento estable; generación interpretativa; control del espacio latente.

Tabla 2. Elementos necesarios para aplicar VAE a datos estructurados. Fuente: elaboración propia.

2.4. Transformers generativos y LLM para datos estructurados

Asociados inicialmente con el modelado secuencial y la generación de datos con atención contextual, los transformers, introducidos por Vaswani *et al.* (2017), marcaron un hito en los modelos generativos, especialmente en el procesamiento del lenguaje natural. Su mecanismo de autoatención (self-attention) permite capturar dependencias complejas y generar salidas condicionadas de alta calidad. A diferencia de las redes neuronales recurrentes (RNN), procesan datos en paralelo y consideran todo el contexto de entrada simultáneamente, lo que mejora la eficiencia y la coherencia en la generación.

En el contexto de generación de datos sintéticos, los transformers ofrecen ventajas únicas frente a VAEs y GANs: no requieren un espacio latente explícito ni una función de pérdida adversarial y permiten generar datos altamente estructurados y condicionados, como secuencias, tablas o código, manteniendo relaciones internas complejas. Estas propiedades han impulsado su uso creciente en tareas como la síntesis de datos tabulares, el modelado temporal y la generación estructurada controlada.

Los modelos generativos basados en transformers, como la familia GPT (Brown *et al.*, 2020), utilizan solo la parte decodificadora de la arquitectura original, adaptada con un mecanismo de autoatención enmascarada (causal). Esta modificación es esencial para garantizar que la generación de datos se realice de forma autoregresiva, es decir, produciendo una salida paso a paso, donde cada nuevo elemento depende únicamente de los anteriores. A diferencia de la autoatención estándar, que permite al modelo acceder a toda la secuencia, la autoatención enmascarada bloquea el acceso a posiciones futuras durante el entrenamiento y la generación.

En la práctica, esto significa que, al generar, por ejemplo, una palabra, un número o un token, el modelo solo puede basarse en los elementos anteriores de la secuencia y no en los que aún no ha producido. Esta restricción es clave para evitar inconsistencias lógicas y asegurar que el modelo aprenda correctamente la probabilidad condicional de cada elemento dado su contexto previo. Gracias a este mecanismo, los transformers pueden generar datos sintéticos que mantienen la coherencia y el orden natural, como ocurre en la creación de texto, las series temporales o las secuencias de registros con dependencia temporal o estructural.

Una de las claves en la generación secuencial de datos es modelar correctamente la distribución conjunta de todos los elementos de una secuencia, es decir, la probabilidad de que aparezca una secuencia completa como $(x_1, x_2, \dots, x_n)$. Para lograrlo, se descompone esta distribución en una serie de distribuciones condicionales según la regla de la cadena de probabilidades:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | x_{1:i-1})$$

Esto significa que la probabilidad de toda la secuencia se calcula multiplicando: uno por uno, la probabilidad de cada elemento x_i condicionada a los anteriores. Por ejemplo, para generar una frase, se empieza estimando $P(x_1)$, luego $P(x_2 | x_1)$, después $P(x_3 | x_1, x_2)$ y así sucesivamente.

Este enfoque es fundamental porque permite generar datos de forma ordenada y coherente, donde cada paso se basa en lo que ya se ha generado. Los Transformers generativos, como GPT, implementan este principio utilizando autoatención enmascarada, que asegura que, al predecir x_i , el modelo solo tenga acceso a $x_1, \dots, x_{i-1}$, nunca a los futuros. Así, se respeta la estructura causal necesaria para que los datos sintéticos generados reflejen relaciones reales en el tiempo o en la secuencia.

Si los datos estructurados se formatean como texto plano o JSON: estos modelos pueden generar filas completas de tablas con estructura coherente.

Aunque las GAN y los VAE están más especializados en datos tabulares clásicos, los LLM (*large language models*) basados en transformers pueden generar datos estructurados si se formatean correctamente; por ejemplo, texto delimitado (*CSV-like*), estructuras anidadas en JSON y cadenas etiquetadas estilo XML. Gracias a su capacidad para comprender y generar texto con coherencia contextual, los LLM, como GPT o LLaMA, desarrollado por Meta (antes Facebook), pueden utilizarse para generar datos sintéticos realistas en tareas que requieren conservar la estructura y el significado. Esto permite, por ejemplo:

- ▶ Simular registros con patrones semánticos, como historiales médicos, líneas de pedido o perfiles de clientes, respetando el formato y las relaciones internas entre los campos.
- ▶ Generar ejemplos sintéticos para documentación o pruebas, es decir, crear textos o datos simulados que imitan el formato y el contenido de entradas reales, pero que no contienen información confidencial ni provienen de usuarios reales. Estos ejemplos son útiles para manuales, tutoriales, demostraciones o pruebas de sistemas, ya que permiten mostrar cómo funciona una aplicación sin comprometer la privacidad.
- ▶ Componer automáticamente entradas de formularios, como solicitudes, encuestas o datos de registro, con contenido coherente y contextualizado.
- ▶ Crear registros de eventos o registros de sistemas simulados, útiles para pruebas de rendimiento, detección de errores o entrenamiento de modelos de análisis.

En resumen, los LLMs permiten automatizar la creación de datos estructurados con sentido, algo especialmente útil cuando se necesita mantener la lógica del contenido sin usar datos reales.

En la tabla tres se resumen los requisitos técnicos para aplicar LLMs a la generación estructurada:

Elemento	Descripción
Formato de entrada	Secuencia textual con estructura (CSV, JSON, XML, YAML, o prosa con formato semántico).
Condicionamiento	Opcional. Se puede usar un <i>prompt</i> con instrucciones o ejemplos previos (few-shot prompting).
Modelo recomendado	LLMs tipo GPT (OpenAI), T5 (Google), LLaMA (Meta), Claude (Anthropic), o modelos open-source disponibles en Hugging Face.
Herramientas / software	OpenAI API, Transformers (Hugging Face), LangChain, PandasAI y Code Interpreter. Permiten integrar o ampliar el uso de modelos de lenguaje para tareas prácticas, como la creación de datos estructurados, automatización de pruebas, análisis asistido por IA y generación de código para simulaciones o transformación de datos.
Tipo de salida esperada	Registros o fragmentos sintéticos con estructura válida y contenido verosímil.
Ventajas	Flexibilidad, generación contextual, integración con lenguaje natural y código.
Limitaciones	Coste computacional, posibilidad de errores de formato o inconsistencias sin postprocesamiento. En muchos casos es necesario aplicar etapas de validación o posprocesamiento para asegurar la calidad y usabilidad de los datos sintéticos generados.

Tabla 3. Requisitos técnicos para aplicar LLM a generación estructurada. Fuente: elaboración propia.

Más allá de la generación directa de datos sintéticos, los LLM ofrecen capacidades complementarias en procesos analíticos complejos como:

- ▶ Generación de código para preprocesamiento: los LLM pueden traducir descripciones en lenguaje natural a código para limpiar, transformar o simular datos (Boonstra, 2024).
- ▶ Imputación de datos faltantes: permiten completar registros incompletos generando valores plausibles basados en el contexto.
- ▶ Conversión texto-tabla y viceversa: facilitan la transformación entre descripciones narrativas y estructuras tabulares; es útil en documentación, interfaces o generación guiada.

Estas aplicaciones no sustituyen a los modelos generativos clásicos (como GAN o VAE), pero los complementan al integrarse en procesos donde intervienen texto, lógica y estructuras semiformales, ampliando así las posibilidades de generación y manipulación de datos sintéticos.

2.5. Comparación de modelos: usos, ventajas y desventajas

A continuación, describimos algunos criterios estratégicos para seleccionar modelos generativos en datos estructurados. La elección no es trivial: cada arquitectura —GAN, VAE o Transformer —sigla LLM— ofrece ventajas diferenciales, limitaciones técnicas y casos de uso óptimos según el objetivo del análisis, la disponibilidad de datos y los requerimientos operativos. Seleccionar una u otra opción depende de múltiples factores.

La Tabla 4 resume las principales características comparativas para facilitar una toma de decisiones fundamentada.

Modelo	Usos principales	Ventajas	Desventajas
GAN (CTGAN)	- Generación de datos realistas. - Aumento de clases minoritarias.	- Alto realismo - Captura relaciones complejas - Generación condicional (Permite generar datos fijando valores en ciertas variables)	- Entrenamiento inestable (sensible a hiperparámetros y arquitectura; puede no converger fácilmente). - Riesgo de colapso de modos (el modelo puede generar solo un tipo de ejemplo, ignorando la diversidad real)
VAE (TVAE)	- Imputación de valores faltantes. - Exploración de variantes sintéticas.	- Entrenamiento estable - Generación diversa (produce una variedad amplia de ejemplos, no repetitivos ni centrados en un solo caso). - Espacio latente interpretable (las variables internas latentes tienen un significado útil para explorar o controlar la salida).	- Muestras menos precisas - Tendencia a promediar características (puede generar ejemplos intermedios que no representan casos reales específicos).
Transformer (LLM)	- Prototipado rápido. - Simulación semántica en texto, JSON y estructuras tabulares (Puede generar nuevos tipos de datos con solo cambiar la instrucción, sin volver a entrenar el modelo).	- Muy flexible (capaz de adaptarse a muchos formatos o tareas con poca configuración). - Interfaz natural (lenguaje). - Sin necesidad de reentrenamiento.	- Menor control estadístico (difícil garantizar que los datos generados sigan una distribución concreta). - Posibles errores de formato o consistencia (puede producir datos mal estructurados o incoherentes entre campos).

Tabla 4. Ventajas y desventajas en modelos generativos. Fuente: elaboración propia.

Aunque las GAN proporcionan datos muy realistas, las VAE permiten un entrenamiento estable y diverso, y las LLM destacan por su generación rápida y flexible guiada por indicaciones naturales. (Guja *et al.*, 2025; Ramchandani, 2025)

2.6. Modelos condicionales y control de generación

En la generación de datos sintéticos no siempre es deseable una salida completamente aleatoria. En muchas aplicaciones reales, como segmentación de mercado, pruebas dirigidas o análisis de escenarios hipotéticos, es crucial poder controlar o condicionar el proceso generativo para obtener muestras que cumplan criterios específicos.

Este control sobre la generación de datos puede aplicarse en distintas etapas del proceso. Durante el entrenamiento, los modelos condicionales permiten guiar la generación al incorporar variables específicas como parte del aprendizaje (por ejemplo, clase, categoría o contexto). En la fase de muestreo, es posible aplicar técnicas de generación dirigida, ajustando parámetros o fijando condiciones para obtener ejemplos con características deseadas. Finalmente, en el caso de los LLMs, el control puede ejercerse a través del diseño del prompt (*prompting estructurado*), formulando instrucciones o plantillas que orienten la generación hacia formatos, estructuras o contenidos específicos.

La tabla 5 muestra un resumen del control de generación en los modelos revisados.

Modelo	Técnica de control	Punto de aplicación	Ejemplo
CTGAN (GAN)	Las variables categóricas de condición se convierten en embeddings vectoriales (capturan relaciones semánticas entre categorías al situarlas en un espacio vectorial en el que la distancia entre vectores refleja similitud o comportamiento relacionado) y se añaden como entrada tanto al generador como al discriminador. Esto permite guiar el tipo de muestra a generar según la categoría deseada. Este enfoque es útil para equilibrar <i>datasets</i> desbalanceados, crear conjuntos por clase o generar casos raros.	Entrenamiento y muestreo	<code>ctgan.sample(10, condition_column='pais', condition_value='DE')</code> . Se generan 10 registros sintéticos específicos para Alemania, útil para simular escenarios localizados.
CVAE (VAE)	La variable condicional se concatena al vector de entrada y también se codifica como <i>embedding</i> para influir la codificación y decodificación. Esto dirige la generación hacia regiones latentes específicas según la condición.	Entrenamiento y generación	Generar registros con <code>tipo='empresarial'</code> al incluir <code>tipo</code> como variable de entrada. Se producen datos sintéticos de clientes del segmento «empresarial», útil para exploración o validación focal. El modelo aprende cómo varía la distribución de datos según el tipo.
LLM (<i>Transformer</i>)	El control se logra mediante <i>prompts</i> diseñados con condiciones explícitas. El modelo interpreta estos textos como restricciones semánticas, sin necesidad de estructura matemática adicional como <i>embeddings</i> . Los LLMs interpretan estas instrucciones y ajustan su distribución de salida en tiempo real. Aunque el control estadístico es más laxo que en modelos explícitamente condicionados, la flexibilidad semántica y expresiva es incomparable.	Entrada (<i>prompt</i>)	«Genera 3 perfiles con ciudad = 'Barcelona' y edad > 30». Se generan descripciones JSON de perfiles sintéticos con atributos específicos, aplicables a pruebas o ejemplos.

Tabla 5. Control de generación en distintos modelos sintéticos. Fuente: elaboración propia.

En resumen, los distintos enfoques generativos ofrecen capacidades complementarias en cuanto al control de la generación sintética. Las GAN y los VAE permiten un control categórico preciso y explicable, ideal cuando se desea condicionar la generación a etiquetas o clases específicas. Los VAE, además, ofrecen una mejor capacidad para manipular variables continuas gracias a su

espacio latente estructurado. Por su parte, los modelos de lenguaje (LLM) destacan por su alta escalabilidad semántica, al permitir generar datos complejos a partir de descripciones en lenguaje natural, aunque con menor precisión estadística y una explicabilidad más dependiente de la interpretación del prompt. En conjunto, la elección del modelo dependerá del tipo de control requerido, de la interpretabilidad deseada y del grado de flexibilidad necesario.

2.7. Evaluación de calidad de datos generados

La generación de datos sintéticos no es útil si los producidos no imitan adecuadamente la realidad o no resultan funcionales en aplicaciones prácticas. Por ello, evaluar la calidad de los datos generados es un componente crítico en todo sistema de IA generativa.

Es fundamental entonces considerar tanto la fidelidad estadística como la utilidad práctica al evaluar datos sintéticos, ya que una no garantiza la otra. La alta fidelidad indica que las distribuciones y relaciones en los datos generados se asemejan a las del conjunto real, pero esto no implica necesariamente que los datos sean útiles. Por ejemplo, si los datos sintéticos son redundantes, triviales o carecen de variabilidad, podrían presentar buen aspecto estadístico, pero no aportar valor en tareas reales. Por el contrario, la alta utilidad práctica, como un buen rendimiento en modelos entrenados con datos sintéticos, no asegura que esos datos sean representativos: podrían reflejar sobreajuste, una generación poco controlada o una representación sesgada de los datos reales. Por ello, ambos enfoques deben considerarse de forma complementaria para garantizar que los datos generados sean tanto realistas como funcionales en el contexto previsto.

La evaluación de la calidad de los datos sintéticos generados debe adaptarse al tipo de modelo utilizado, ya que cada enfoque produce los datos de forma distinta y, por tanto, requiere criterios específicos. En el caso de modelos basados en GAN, como CTGAN, se utilizan métricas estadísticas clásicas, como la divergencia de Kullback-Leibler o la distancia de Wasserstein, además de análisis de correlaciones y visualizaciones en espacios reducidos. Los VAE, al generar datos a partir de un espacio latente comprimido, permiten evaluar no solo si los datos generados se parecen a los reales, sino también qué tan bien reconstruyen los datos originales, qué variedad de ejemplos pueden producir y si los datos generados conservan las

estructuras naturales presentes en los datos reales; por ejemplo, mediante análisis de agrupamiento (evaluar si los datos sintéticos conservan patrones de grupos o clases similares a los reales). En cambio, los LLM, al operar sobre texto y estructuras como JSON o formularios, requieren evaluaciones centradas en la validez del formato, la coherencia semántica y la funcionalidad de la salida generada. La tabla siguiente resume las métricas y métodos más adecuados para cada enfoque, agrupados en las dos dimensiones clave mencionadas.

Para que la evaluación sea robusta y progresiva, se recomienda comenzar con métodos tradicionales —fáciles de interpretar y familiares para cualquier analista— y, en una segunda etapa, aplicar métricas más avanzadas que permitan un análisis más preciso de la similitud entre distribuciones. Esta combinación asegura comprensión y rigor.

Las métricas tradicionales, como medias y varianzas, permiten comprobar si los valores centrales y la dispersión de las variables sintéticas se asemejan a los reales. Las matrices de correlaciones evalúan si las relaciones lineales entre variables se conservan y las pruebas de hipótesis, como Kolmogorov–Smirnov (para distribuciones continuas) y ji cuadrado (para categóricas), permiten verificar si las diferencias son significativas. Estas métricas permiten una primera validación rápida y accesible.

Por su parte, las métricas más especializadas capturan diferencias más sutiles en las distribuciones y patrones complejos.

Antes de ver las fórmulas, es fundamental definir sus componentes principales a nivel empírico:

- ▶ P : representa la distribución de probabilidad de los datos reales.
- ▶ Q : representa la distribución de probabilidad de los datos sintéticos.

- ▶ p_i : es la probabilidad o frecuencia relativa de la categoría i en los datos reales (P).
- ▶ q_i : es la probabilidad o frecuencia relativa de la categoría i en los datos sintéticos (Q).

Divergencia de Jensen–Shannon (JSD): mide la similitud entre dos distribuciones de probabilidad, la distribución real (P) y la sintética (Q). Al ser simétrica y acotada (usualmente entre 0 y 1), es muy intuitiva. Un valor cercano a 0 indica una alta fidelidad entre la distribución real y la sintética.

$$JSD(P,Q) = \frac{1}{2} \sum_i p_i \log \frac{p_i}{m_i} + \frac{1}{2} \sum_i q_i \log \frac{q_i}{m_i}$$

Con

$$m_i = \frac{1}{2}(p_i + q_i)$$

La **distancia de Wasserstein** (1D) compara los valores ordenados de dos distribuciones. Su gran ventaja es que proporciona una medida significativa de la distancia incluso cuando las distribuciones no se superponen. Para el caso práctico de dos muestras con el mismo número de puntos, se calcula comparando sus valores una vez ordenados (es decir, sus cuantiles empíricos). Cuanto menor sea la distancia, más semejantes son las distribuciones en su forma y dispersión. Es decir, la distancia de Wasserstein entre la distribución real P y la sintética Q se calcula tomando la lista de datos reales x y la lista de datos sintéticos y , ordenándolas y calculando la media de las diferencias absolutas entre sus valores emparejados. Esta medida refleja de manera intuitiva cuánto «trabajo» se requiere para transformar una distribución en otra y resulta especialmente adecuada para evaluar la similitud entre distribuciones reales y sintéticas. La fórmula para datos en una dimensión es:

$$d_W(P,Q) = \frac{1}{n} \sum_i^n |x_{(i)} - y_{(i)}|$$

Distancia de Variación Total (TVD): Mide la máxima discrepancia en la probabilidad que dos distribuciones asignan al mismo evento o categoría. Es simple de interpretar y su valor siempre está entre 0 (idénticas) y 1 (totalmente diferentes). Un valor de $TVD = 0.1$ implica que existe un 10% de diferencia global en las proporciones entre las dos distribuciones. La fórmula es:

$$TVD(P,Q) = \frac{1}{2} \sum_i p_i - q_i$$

En resumen: TVD y JSD se aplican sobre distribuciones de probabilidad (p,q) , lo que resulta natural en datos categóricos o discretizados. Wasserstein se aplica sobre las distribuciones acumuladas o sus cuantiles, lo que, en la práctica, equivale a comparar las listas ordenadas de valores numéricos (x,y) .

La tabla 6 muestra una guía escalonada de evaluación: los indicadores estadísticos tradicionales proporcionan una primera validación básica e intuitiva, mientras que las métricas avanzadas capturan diferencias más sutiles entre distribuciones. Ambas aproximaciones son complementarias y, juntas, aseguran que los datos sintéticos se evalúen de manera comprensible y rigurosa. La tabla se lee en dos niveles complementarios. Los indicadores tradicionales (medias, varianzas, correlaciones, histogramas, pruebas sencillas) permiten una primera validación intuitiva y accesible. Posteriormente, las métricas más avanzadas (JSD, Wasserstein, TVD, visualizaciones latentes) ofrecen un análisis más detallado y robusto. En todos los casos, la utilidad práctica se valida con experimentos aplicados (TSTR, TRTS, métricas predictivas, reglas de negocio). Este enfoque escalonado asegura que los datos sintéticos se evalúen tanto en su fidelidad estadística como en su aplicabilidad real.

Modelo	Fidelidad estadística	Utilidad práctica
CTGAN (GAN)	Indicadores tradicionales: medias, varianzas, correlaciones, <i>test</i> KS, chi-cuadrado. Avanzados: Wasserstein Distance, KL <i>divergence</i> , total variation distance (TVD), visualización en PCA/UMAP.	TSTR (Train on Synthetic, Test on Real), TRTS (Train on Real, Test on Synthetic), métricas predictivas (Accuracy, F1), pruebas de ablación.
TVAE (VAE)	Indicadores tradicionales: histogramas, <i>boxplots</i> , correlaciones condicionales. Avanzados: Jensen–Shannon Divergence, visualización en espacio latente (UMAP, t-SNE).	TSTR / TRTS, <i>clustering</i> sintético, evaluación de reconstrucción (MSE, MAE), transferencia de representaciones latentes.
LLM (Transformers)	Indicadores tradicionales: validación estructural (tipos de datos, formatos), reglas básicas de consistencia. Avanzados: validación semántica contextual, QA automático, testeo guiado.	Evaluación funcional (cumplimiento del propósito), testeo automatizado, validación con reglas de negocio, revisión manual asistida por <i>scripts</i> .

Tabla 6. Evaluación de calidad de datos sintéticos por tipo de modelo. Fuente: elaboración propia.

En la práctica, no existe una única métrica capaz de captar todos los aspectos implicados en la calidad de los datos sintéticos. Por tanto, se recomienda adoptar una estrategia de evaluación multidimensional que combine indicadores tanto de fidelidad estadística como de utilidad práctica, ajustados al objetivo del proyecto y a la arquitectura empleada.

Como ejemplo, para ilustrar la aplicación de estas métricas, consideremos tres variables:

- ▶ Datos reales (R):
 - Tipo_cliente: 6 fidelizados y 4 nuevos.
 - Edad: [25, 28, 30, 32, 35, 36, 38, 40, 42, 45].
 - Ingreso: [22, 25, 28, 30, 33, 34, 36, 39, 40, 44].
- ▶ Datos sintéticos (S):
 - Tipo_cliente: 5 fidelizados y 5 nuevos.
 - Edad: [26, 27, 31, 33, 34, 37, 39, 41, 43, 46].
 - Ingreso: [23, 26, 29, 31, 32, 35, 37, 38, 41, 45].

Aplicación de métricas

- ▶ Medias y varianzas (edad): reales = 35.1 (var = 45.2), sintéticos = 35.7 (var = 37.6) → valores próximos.
- ▶ Correlación (edad–ingreso): reales = 0.98, sintéticos = 0.97 → se preserva la relación lineal.
- ▶ Chi-cuadrado (tipo_cliente): 6 vs. 4 frente a 5 vs. 5 → p-valor > 0.5 → sin diferencias significativas.
- ▶ Kolmogorov–Smirnov (edad): $D = 0.2$, $p \approx 0.8$ → distribuciones similares.
- ▶ JSD (tipo_cliente): $P=(0.6,0.4)$, $Q=(0.5,0.5)$ → $JSD \approx 0.009$ → muy similares.

- ▶ Wasserstein (edad): ≈ 0.9 años $\rightarrow$ se necesita desplazar menos de 1 año promedio para alinear ambas distribuciones.
- ▶ TVD (tipo_cliente): 0.1 $\rightarrow$ 10 % de discrepancia global en proporciones.

«Este conjunto de métricas muestra cómo los datos sintéticos reproducen adecuadamente según las estadísticas tre básicas (medias, varianzas, correlaciones) como en las relaciones globales (JSD, Wasserstein, TVD)».

Por otro lado, una de las mejores prácticas consiste en aplicar métricas de fidelidad y utilidad de forma complementaria. Por ejemplo, un conjunto sintético puede mostrar una distribución de ingresos muy similar a la del conjunto real (alta fidelidad), pero si no mejora el rendimiento de un modelo predictivo, su utilidad es limitada. Del mismo modo, un conjunto de datos puede mejorar la precisión de un modelo, pero si introduce valores estadísticamente improbables o inconsistencias internas, su fidelidad y confiabilidad quedan comprometidas.

Otro aspecto fundamental consiste en prestar especial atención a las variables sensibles, como el sexo, la raza, los ingresos o la discapacidad. Estas variables deben evaluarse por separado para identificar posibles sesgos, omisiones o representaciones distorsionadas que podrían derivar en consecuencias éticas o legales. A nivel práctico, resulta imprescindible acompañar cualquier análisis cuantitativo con una inspección visual cuidadosa: histogramas, análisis de componentes principales (PCA), diagramas de dispersión y representaciones en UMAP son herramientas eficaces para detectar anomalías o inconsistencias estructurales.

Finalmente, se debe garantizar la coherencia lógica de los datos generados. Por ejemplo, un perfil que combine una edad de 14 años con la profesión de cirujano, aunque pueda ser posible desde una perspectiva combinatoria, carece de plausibilidad real. Este tipo de validación semántica es especialmente importante en

contextos sensibles o de toma de decisiones. Asimismo, siempre que los datos sintéticos se utilicen con fines analíticos, regulatorios o comerciales, es recomendable documentar detalladamente el proceso de evaluación, incluidas las métricas utilizadas, las herramientas aplicadas y los criterios de aceptación o rechazo adoptados.

En conjunto, una evaluación bien diseñada y rigurosamente ejecutada no solo mejora la calidad técnica de los datos generados, sino que también fortalece su confiabilidad, trazabilidad y valor real en entornos profesionales.

2.8. Exploración práctica

Los modelos generativos abordados en este tema permiten no solo crear datos sintéticos, sino también simular escenarios de negocio, aumentar muestras y generar estructuras coherentes que antes requerían codificación manual. En este apartado se proponen dos casos prácticos que aplican las arquitecturas presentadas (GAN y transformadores), con énfasis en la generación y evaluación de datos tabulares sintéticos, el uso de prompting estructurado para generación controlada y la integración práctica de métricas y buenas prácticas.

Caso Práctico1: Simulación de perfiles de cliente con CTGAN (GAN)

Un equipo de marketing desea analizar cómo responderían clientes potenciales alemanes a una nueva campaña. Por razones legales y técnicas, no puede usar datos reales. Necesita simular perfiles sintéticos que reflejen la realidad demográfica y financiera del país.

Objetivo: Generar perfiles sintéticos de clientes europeos para pruebas de segmentación y entrenamiento de modelos de marketing.

Modelo: CTGAN, Conditional GAN para datos tabulares mixtos (numéricos y categóricos). Implementación a través de la librería SDV, desarrollada por el MIT Data to AI Lab.

Nota: El paquete ctgan es parte de **SDV**, una suite de herramientas de código abierto desarrollada por el **MIT Data to AI Lab**, diseñada para generar, modelar y evaluar datos sintéticos. La suite incluye también TVAE, GaussianCopula, SDMetrics, y utilidades de validación. Se puede instalar con:

```
pip install sdv
```

CODE. Implementación

```
from ctgan import CTGAN
import pandas as pd

# Datos semilla
data = pd.DataFrame({
    "edad": [34, 28, 45, 39, 30],
    "pais": ["DE", "FR", "DE", "ES", "ES"],
    "ingresos_mensuales": [2500, 2100, 3200, 2800, 2300],
    "tipo_cliente": ["nuevo", "nuevo", "fidelizado", "fidelizado", "nuevo"]
})
categorical = ["pais", "tipo_cliente"]

# Entrenamiento del modelo
ctgan = CTGAN(epochs=300)
ctgan.fit(data, categorical)

# Generación condicional
synthetic = ctgan.sample(10, conditions={"pais": "DE"})
print("Datos sintéticos generados (condición: pais='DE'):")
print(synthetic)
```

Fidelidad estadística

Se comparan las medias, las varianzas y las correlaciones clave con los datos reales alemanes. Luego, se visualiza la proximidad en un espacio reducido con el análisis de componentes principales.

```
# Comparación visual en espacio reducido (PCA)
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Datos reales de Alemania
real = data[data["pais"] == "DE"]

# Unir reales y sintéticos etiquetando origen
joined = pd.concat([
    real.assign(origen="Real"),
    synthetic.assign(origen="Sintético")
], ignore_index=True)

# PCA sobre variables numéricas
pca_input = joined[["edad", "ingresos_mensuales"]]
pca = PCA(n_components=2).fit_transform(pca_input)
joined["PCA1"], joined["PCA2"] = pca[:, 0], pca[:, 1]

# Visualización
plt.figure(figsize=(8, 6))
sns.scatterplot(data=joined, x="PCA1", y="PCA2", hue="origen", style="origen", s=100)
plt.title("Proximidad entre Datos Reales y Sintéticos (PCA)")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Utilidad práctica

Entrenamos un modelo predictivo en los datos sintéticos, y lo validamos en los reales. Evaluamos su rendimiento:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Variables predictoras y objetivo
X_synth = pd.get_dummies(synthetic.drop(columns="tipo_cliente"))
y_synth = synthetic["tipo_cliente"]

X_real = pd.get_dummies(real.drop(columns="tipo_cliente"))
y_real = real["tipo_cliente"]

# Alinear columnas para asegurar compatibilidad
X_synth, X_real = X_synth.align(X_real, join='left', axis=1, fill_value=0)
```



```
# Entrenamiento y predicción
model = RandomForestClassifier(random_state=42)
model.fit(X_synth, y_synth)
preds = model.predict(X_real)

# Evaluación
print("\n=== Evaluación TSTR (Train on Synthetic, Test on Real) ===")
print("Accuracy:", accuracy_score(y_real, preds))
print("\nReporte de clasificación:\n", classification_report(y_real, preds))
```

Este caso muestra cómo entrenar un modelo GAN (CTGAN) para generar datos tabulares realistas a partir de un pequeño conjunto semilla con capacidad de capturar tanto variables categóricas como numéricas. A través de la librería SDV («Data To AI Lab del MIT»), se construye un generador condicional que permite simular perfiles de cliente nuevos manteniendo las propiedades estadísticas del conjunto original.

Además, se abordan dos ejes clave de validación. Por un lado, la fidelidad estadística se comprueba comparando distribuciones («TVD»), correlaciones y proyecciones visuales con PCA. Por otro lado, se evalúa la utilidad práctica del dataset sintético usando el enfoque «TSTR», entrenando un modelo predictivo sobre los datos generados y validándolo con datos reales.

Caso práctico 2: generación estructurada de eventos con LLM (Transformer)

Un equipo de aseguramiento de la calidad (QA, por sus siglas en inglés) necesita simular eventos de usuario como inicio de sesión, compra y cierre de sesión en formato JSON para probar una API. Dado que no puede utilizar trazas reales, requiere una solución rápida, flexible y que permita controlar el contenido generado.

Objetivo: Usar un LLM como generador de eventos JSON estructurados para simular el tráfico de usuarios en una aplicación.

Modelo: Modelo generativo tipo GPT (Transformer decodificador), controlado por prompting few-shot.

En términos didácticos, este caso práctico persigue un doble objetivo. Se reproduce un escenario común en aseguramiento de la calidad (QA): generar eventos de usuario en formato JSON para probar una API sin exponer trazas reales ni datos sensibles. Es decir, se crean datos de prueba que parecen reales, pero son falsos, para poner a prueba un sistema. En el plano didáctico, busca mostrar al estudiante cómo se complementan dos enfoques distintos en la generación de datos estructurados. El primero, basado en prompting con un LLM, ilustra la capacidad de los modelos de lenguaje para crear secuencias de manera rápida y flexible. El segundo, basado en una simulación programada en Python, ofrece un entorno controlado para validar la estructura, la coherencia temporal y la consistencia semántica de los datos. De esta forma, se comprende tanto el potencial creativo de la IA generativa como la importancia de contar con herramientas sistemáticas de validación.

Prompt few-shot

Debes generar eventos JSON para simular la actividad de usuarios en una app móvil.

Con base en los siguientes ejemplos, genera una lista de 10 eventos similares. Usa identificadores distintos, tiempos realistas y acciones posibles, como: «login», «view», «purchase», «logout».

Ejemplos:

```
[
  {
    "event_id": "evt_001",
    "timestamp": "2025-07-01T10:00:00Z",
    "user_id": "U01",
    "action": "login"
  },
  {
    "event_id": "evt_002",
    "timestamp": "2025-07-01T10:02:12Z",
    "user_id": "U01",
    "action": "purchase"
  }
]
```

Tu respuesta debe ser un bloque «JSON» válido con diez nuevos objetos de evento, respeta la estructura, los tipos de datos y genera secuencias lógicas de acciones por usuario.

Este prompt aplica los principios de *prompt engineering estructurado* de Boonstra (2024), como:

Rol funcional claro: «Eres un generador de eventos JSON...»

Ejemplos bien formateados: dos eventos completos como guía del formato esperado.

Control semántico por instrucciones: se limita el dominio de valores y se pide coherencia temporal.

Asignar un rol al modelo y mostrar ejemplos precisos permite controlar tanto el formato como el contenido de la generación (Boonstra, 2024).

Generación de eventos

```
from datetime import datetime, timedelta
import random
import json

# Configuración de usuarios y acciones posibles
usuarios = ["U01", "U02", "U03"]
acciones_posibles = ["login", "view", "purchase", "logout"]
eventos = []
event_id_counter = 1
base_time = datetime(2025, 7, 1, 10, 0, 0)

# Generación de secuencias de eventos por usuario
for usuario in usuarios:
    time = base_time
    acciones = ["login"] + random.choices(["view", "purchase"], k=random.randint(1, 3)) + ["logout"]
    for accion in acciones:
        eventos.append({
            "event_id": f"evt_{event_id_counter:03}",
            "timestamp": time.isoformat() + "Z",
            "user_id": usuario,
            "action": accion
        })
        time += timedelta(seconds=random.randint(30, 300))
        event_id_counter += 1

# Imprimir como JSON válido
print(json.dumps(eventos, indent=2))
```

Estructura válida

Se verifica que todos los objetos contengan los campos esperados con tipos de datos válidos.

```
import pandas as pd

# Convertir eventos a DataFrame
df = pd.DataFrame(eventos)
```

```
# Validar estructura exacta
for i, e in enumerate(eventos):
    assert set(e) == {"event_id", "timestamp", "user_id", "action"}, f"Estructura incorrecta en evento {i}"

print("Todos los eventos tienen estructura válida.")
```

Semántica y coherencia Se valida que la secuencia de acciones tenga lógica (por ejemplo, no debe haber *logout* sin *login*):

```
# Validar coherencia de acciones por usuario
for user in df["user_id"].unique():
    user_events = df[df["user_id"] == user].sort_values("timestamp")
    actions = list(user_events["action"])

    if "logout" in actions and "login" not in actions:
        print(f"Inconsistencia: logout sin login para {user}")
    if "purchase" in actions and "login" not in actions:
        print(f"Inconsistencia: purchase sin login para {user}")
    if "login" in actions and "logout" in actions:
        if actions.index("logout") < actions.index("login"):
            print(f"Inconsistencia: logout antes de login para {user}")

print("Validación semántica completada.")
```

Conversión a tabla

El JSON generado se interpreta como tabla y se visualizan patrones:

```
Import seaborn as sns
import matplotlib.pyplot as plt

# Visualización de frecuencia de acciones
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="action", order=["login", "view", "purchase", "logout"])
plt.title("Distribución de acciones en eventos simulados")
plt.ylabel("Frecuencia")
plt.xlabel("Acción")
plt.tight_layout()
plt.show()
```

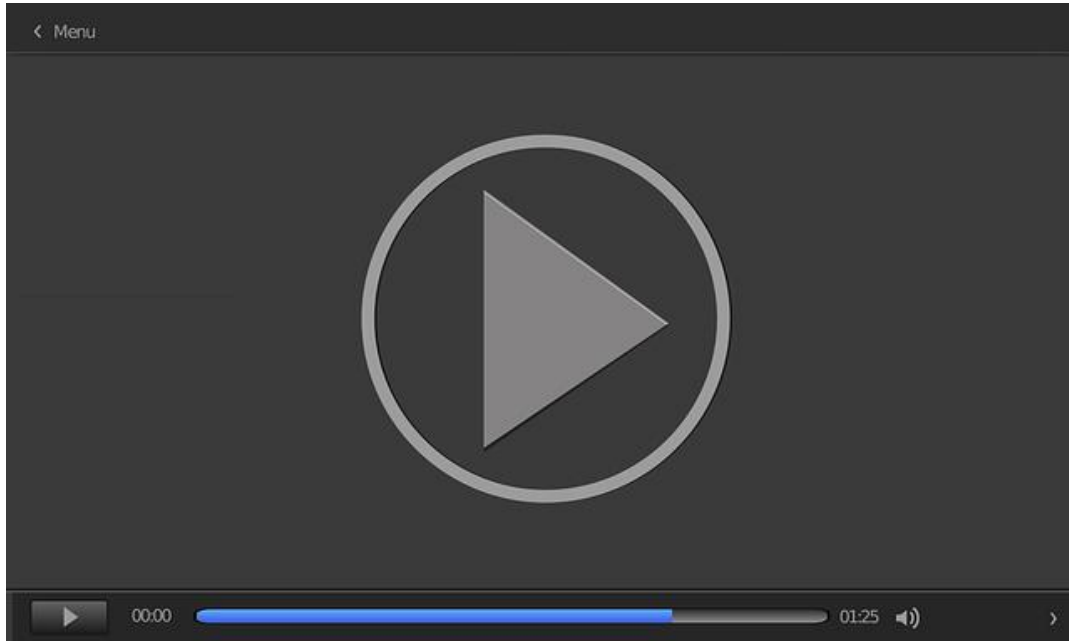
Este caso muestra cómo un modelo de lenguaje (LLM), como los basados en Transformers, puede ser utilizado para generar datos estructurados en

formato JSON, donde cada objeto representa una “fila” en términos tabulares. La generación se controla íntegramente mediante *prompting* en lenguaje natural, sin necesidad de entrenamiento adicional. A través de un prompt bien diseñado (con ejemplos, restricciones y objetivos claros), se consigue generar múltiples registros consistentes que respetan estructura, dominio de valores y lógica temporal.

La evaluación de los datos generados se realiza como si se tratara de un dataset tabular: se revisa la validez del formato (por ejemplo, campos presentes, tipos correctos), la coherencia semántica (acciones compatibles con la secuencia de eventos por usuario) y la distribución de valores.

Durante la elaboración del tema, se empleó un modelo de IA generativa como herramienta de apoyo para optimizar la redacción y precisar conceptos técnicos. Su uso se llevó a cabo con responsabilidad y criterio académico, mediante múltiples iteraciones de consulta, análisis y reformulación que, gracias a la debida experticia, permitieron ajustar las respuestas con rigor y precisión al contexto específico del contenido desarrollado (OpenAI, 2025). Este uso complementario no sustituyó la consulta de fuentes académicas ni el ejercicio intelectual de construcción, sino que buscó reforzar la calidad y la claridad en el marco de una práctica académica transparente y ética.

En el vídeo, *Generación de datos tabulares con GANs y LLMs*, veremos una demostración de cómo usar GANs para generar datos sintéticos estructurados y compararlos con enfoques basados en LLMs. Incluye visualización y validación estadística de la calidad de los datos generados.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=5cd1a401-9b40-458d-9853-b375016127e1>

2.9. Referencias bibliográficas

Boonstra, L. (2024). *Prompt engineering*. Google. <https://archive.org/details/22365-3-prompt-engineering-v-7/mode/2up>

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I. y Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>

Foster, D. (2019). *Generative deep learning: teaching machines to paint, write, compose, and play*. O'Reilly Media. https://books.google.com.ar/books?id=RqegDwAAQBAJ&printsec=frontcover&hl=es&source=gbs_ge_summary_r&cad=0#v=onepage&q&f=false

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. y Bengio, Y. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 27. <https://arxiv.org/abs/1406.2661>

Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press. https://www.researchgate.net/publication/320703571_Ian_Goodfellow_Yoshua_Bengio_and_Aaron_Courville_Deep_learning_The_MIT_Press_2016_800_pp_ISBN_0262_035618

Guja, A., Siwiak, M. y Siwiak, M. (2025). *Starting data analytics with generative AI and Python*. Manning Publications.

Kingma, D. P. y Welling, M. (2014). Auto-encoding variational Bayes. *International Conference on Learning Representations*. <https://arxiv.org/abs/1312.6114>

OpenAI (2025). ChatGPT (Jun 25 version) [large language model]. <https://chat.openai.com/>.

Página de Synthetic Data Vault (SDV). <https://sdv.dev>.

Ramchandani, T. (2025). *A Generative Journey to AI: mastering the foundations and frontiers of generative deep learning*. BPB Publications.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., y Polosukhin, I. (2017). Attention is all you need. *Conference on Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>

Xu, L., Skoularidou, M., Cuesta-Infante, A. y Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. *Conference on Neural Information Processing Systems NeurIPS*. <https://arxiv.org/pdf/1907.00503>

SDV: Welcome to the SDV!

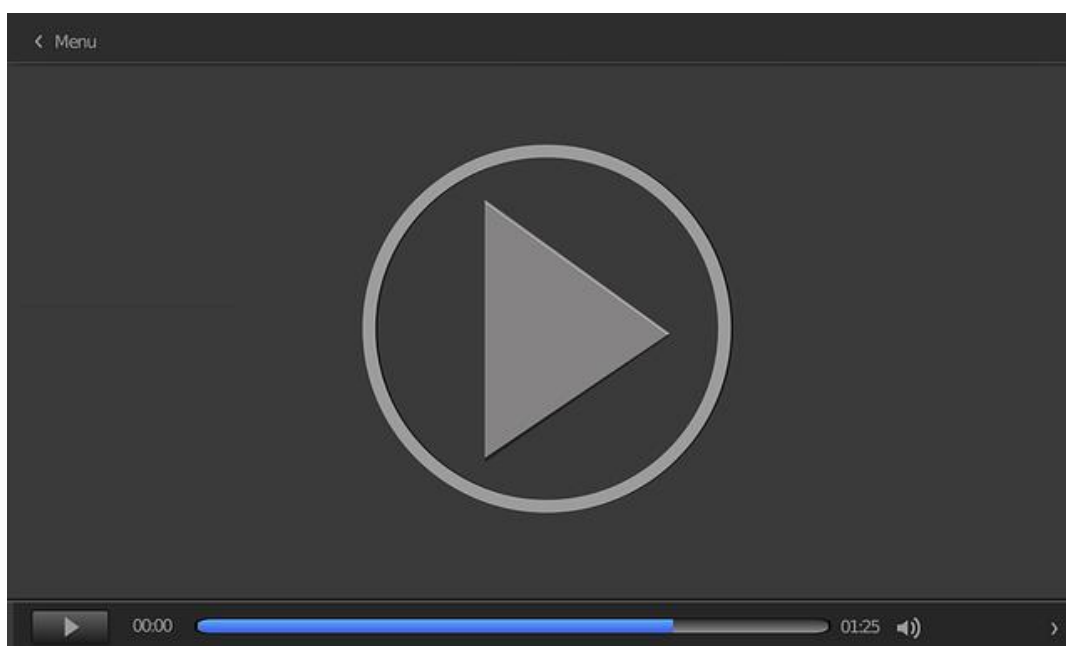
DataCebo, Inc. (2023). *Synthetic Data Vault (SDV)*. https://docs.sdv.dev/sdv?utm_source=chatgpt.com

La documentación oficial de SDV (MIT Data to AI Lab) explica de forma clara cómo generar y evaluar datos sintéticos tabulares. Incluye ejemplos completos con CTGAN y herramientas de análisis, lo que permite al estudiante explorar métricas y visualizaciones tras la generación.

Synthetic Data Generation with CTGAN – Step-by-Step Tutorial

Next Phase with Moe. (2022, octubre 23). *Synthetic data generation with CTGAN* [Video]. YouTube. <https://www.youtube.com/watch?v=Ei0klF38CNs>

Este vídeo muestra cómo generar datos sintéticos tabulares utilizando el modelo CTGAN mediante la API del proyecto Synthetic Data Vault (SDV). A lo largo del tutorial, se explican paso a paso la preparación del entorno, la carga y análisis del *dataset*, la definición de variables categóricas, el entrenamiento del generador y la generación de nuevas muestras. También se incluye la evaluación de los datos sintéticos mediante visualizaciones y métricas estadísticas.



Accede al vídeo:

<https://www.youtube.com/embed/Ei0klF38CNs>

Synthetic data generation using large language models

Nadas, M., Diosan, L. y Tomescu, A. (2025). Synthetic data generation using large language models: Advances in text and code. *IEEE Access*. <https://arxiv.org/pdf/2503.14023?>

Este artículo resume avances en generación de datos sintéticos con LLMs, destacando técnicas de *prompting*, autorefinamiento y métodos para mitigar sesgos e imprecisiones, como el filtrado y la retroalimentación reforzada.